

Report

Task1:

把檔案中的數值都讀取出來到對應的名稱。

```
fid = fopen('network.pos', 'r');
line1 = textscan(fid, '%f %f %f %f %f', 1);
P_tx_dBm = line1{1};
N0_dBm = line1{2};
freq = line1{3};
BW = line1{4};
P_rx_th_dBm = line1{5};

line2 = textscan(fid, '%d %d', 1);
numGS = line2{1};
numSAT = line2{2};
```

把 GS 跟衛星的編號與 XYZ 座標讀取出來，並存到對應的矩陣中。

```
GS_ids = cell(numGS,1);
GS_xyz = zeros(numGS,3);
for i = 1:numGS
    t = textscan(fid, '%s %f %f %f', 1);
    GS_ids{i} = t{1}{1};
    GS_xyz(i,:) = [t{2}, t{3}, t{4}];
end

SAT_ids = cell(numSAT,1);
SAT_xyz = zeros(numSAT,3);
for i = 1:numSAT
    t = textscan(fid, '%s %f %f %f', 1);
    SAT_ids{i} = t{1}{1};
    SAT_xyz(i,:) = [t{2}, t{3}, t{4}];
end
fclose(fid);
```

之後用雙重迴圈來跑所有的 GS 與衛星的組合，算 SNR、data rate 等地方法都跟 lab2 一樣就不再放上來。我是用 sin 去算，算 0-90 度，最後把結果輸出到檔案，依序為 GS 數量、衛星數量、連線數量、之後幾行是 GS 編號 SAT 編號與對應的 data rate

```
for g = 1:numGS
    for s = 1:numSAT
        tx_location = GS_xyz(g,:);
        rx_location = SAT_xyz(s,:);
```

其中加入要大於等於 threshold 才能傳輸成功，才會輸出到結果檔。

```
if Pr_dBm >= P_rx_th_dBm
```

Task2:

先把 task1 做出來的檔案讀取進來，前三個參數分別是 ground station 的數量、衛星的數量、全部 GS 連結到衛星的數量。把 graph 中每一行讀取出來，存到一個矩陣 weight[x][y]，x 代表的是 GS 的編號、y 代表的是衛星的編號、裡面的值是傳輸時間用 $1000 / \text{data rate}$ 算。

```
std::ifstream fin("network.graph");
int num_stations, num_sats, num_links;
fin >> num_stations >> num_sats >> num_links;
const double DATA_UNIT = 1000.0;

std::vector<std::vector<double>> weight(num_stations, std::vector<double>(num_sats, -1.0));
for (int e = 0; e < num_links; ++e) {
    int v, s;
    double rate_kbps;
    fin >> v >> s >> rate_kbps;
    weight[v][s] = DATA_UNIT / rate_kbps;
}
fin.close();
```

利用 or-tools 中的 CBC (Coin-OR Branch and Cut) 來建立 solver 解決 ILP model。建立一個矩陣 x[v][s]，v 是 GS 的編號、s 是衛星的編號、裡面存的是 GS 跟衛星有沒有連線。

```
std::unique_ptr<MPSolver> solver(MPSolver::CreateSolver("CBC_MIXED_INTEGER_PROGRAMMING"));
std::vector<std::vector<const MPVariable*>> x(num_stations, std::vector<const MPVariable*>(num_sats, nullptr));
for (int v = 0; v < num_stations; ++v) {
    for (int s = 0; s < num_sats; ++s) {
        if (weight[v][s] > 0) {
            x[v][s] = solver->MakeBoolVar(
                absl::StrFormat("x_%d_%d", v, s));
        }
    }
}
```

在 solver 中加入變數 T 用來存最大傳輸時間，上界不限、下界是 0。在 solver 中加入每一個 GS 只能連接到一個衛星的限制。對於每一個 GS，x[v][s]總和只能等於一。

```
const MPVariable* T = solver->MakeNumVar(0.0, std::numeric_limits<double>::infinity(), "makespan");
for (int v = 0; v < num_stations; ++v) {
    LinearExpr sum_x;
    for (int s = 0; s < num_sats; ++s) {
        if (x[v][s]) sum_x += x[v][s];
    }
    solver->MakeRowConstraint(sum_x == 1.0);
}
```

對於每一個衛星，算出全部與 GS 連接的時間，用 $x[v][s]$ 乘上 $weight[v][s]$ 代表有沒有連線乘上時間。並且讓時間小於等於最長傳輸時間 T 。取得 solver 的 objective，最小化 $1.0 * T$ 的值，找出 T 的最小可能，即最小傳輸時間。

```
for (int s = 0; s < num_sats; ++s) {
    LinearExpr load;
    for (int v = 0; v < num_stations; ++v) {
        if (x[v][s]) {
            load += weight[v][s] * LinearExpr(x[v][s]);
        }
    }
    solver->MakeRowConstraint(load <= T);
}

MPObjective* const objective = solver->MutableObjective();
objective->SetCoefficient(T, 1.0);
objective->SetMinimization();
```

求解 solver 並把算出來的最小時間印出來。建立 vector，Assign[v]，v 代表 GS 的編號，裡面存的是對應連接的衛星編號，sat_time[s]，s 代表是衛星的編號，裡面存的是此衛星傳輸的時間，如果 $x[v][s]$ 等於一代表有連線。

```
const MPSolver::ResultStatus result_status = solver->Solve();
LOG(INFO) << "Optimal makespan = " << T->solution_value();
std::vector<int> assign(num_stations, -1);
std::vector<double> sat_time(num_sats, 0.0);
for (int v = 0; v < num_stations; ++v) {
    for (int s = 0; s < num_sats; ++s) {
        if (x[v][s] && x[v][s]->solution_value() == 1.0) {
            assign[v] = s;
            sat_time[s] += weight[v][s];
        }
    }
}
```

把算出來的值輸出，先輸出最短時間，每個 GS 對應到的衛星編號，每個衛星編號的傳輸時間，全部輸出到 network.ortools.out。

```
std::ofstream fout("network.ortools.out");
fout << T->solution_value() << "\n";
for (int v = 0; v < num_stations; ++v) {
    fout << v << " " << assign[v] << "\n";
}
for (int s = 0; s < num_sats; ++s) {
    fout << s << " " << sat_time[s] << "\n";
}
}
}

int main(int argc, char* argv[]) {
    InitGoogle(argv[0], &argc, &argv, true);
    absl::SetFlag(&FLAGS_stderrthreshold, 0);
    operations_research::Task2OrTools();
    return EXIT_SUCCESS;
}
```

Task3:

前面跟 task2 方法一樣，先把 graph 的資料讀出來，並計算每一個連接的 transmission time。Assign 存 GS 對應的衛星編號、sat_time 存每個衛星的總傳輸時間，對每個 GS 找出最大的 data rate，並直接選取這條傳輸，並對全部的傳輸時間計算出全部衛星傳輸的最長時間。後面也跟 task2 的方法一樣依序輸出檔案到 network.greedy.out。

```
const double DATA_UNIT = 1000.0;
std::vector<int> assign(num_stations, -1);
std::vector<double> sat_time(num_sats, 0.0);

for (int v = 0; v < num_stations; ++v) {
    double best_rate = -1.0;
    int best_s = -1;
    for (int s = 0; s < num_sats; ++s) {
        if (data_rate[v][s] > best_rate) {
            best_rate = data_rate[v][s];
            best_s = s;
        }
    }
    assign[v] = best_s;
    double t = DATA_UNIT / best_rate;
    sat_time[best_s] += t;
}
```

算出 10 個衛星中最長的傳輸時間

```
double makespan = 0.0;
for (double t : sat_time) {
    if (t > makespan) makespan = t;
}
```

Compare:

左邊是 OPT 的結果，可以發現每個衛星都只有兩個 GS 連接，時間幾乎都差不多，有做良好的平均分配，傳輸時間是 greedy 的三倍快，優點是傳輸時間平均分配非常快，缺點是整體的求解過程比較複雜。右邊的是 greedy 的結果，可以發現每個 GS 都選傳輸時間最短的衛星的話，會有很多 GS 都選取到相同衛星的情況，像編號 8 的衛星就有 6 個 GS 選取到，導致該衛星傳輸時間非常長，而總體時間是用最長的時間來看，導致整體的傳輸時間跟著變非常長。其中也有衛星沒有被選擇的情形，導致衛星產生 idle。優點是求解過程非常快，只要對每個 GS 跑可以連接的衛星就好，缺點是可能會有 GS 選取到相同衛星的情況，導致整體傳輸時間變很長。

0.0541074	0.155601
0 0	0 1
1 0	1 2
2 1	2 2
3 1	3 3
4 2	4 3
5 4	5 3
6 2	6 3
7 3	7 4
8 4	8 5
9 7	9 6
10 3	10 4
11 5	11 5
12 6	12 6
13 8	13 8
14 9	14 8
15 8	15 8
16 7	16 8
17 6	17 8
18 9	18 8
19 5	19 5
0 0.0541	0 0
1 0.0541074	1 0.0261526
2 0.0538256	2 0.052711
3 0.0538077	3 0.104141
4 0.0536517	4 0.0520519
5 0.0523843	5 0.0787247
6 0.0541033	6 0.0520796
7 0.0534636	7 0
8 0.0517569	8 0.155601
9 0.0524782	9 0

Initial commit 我上傳的 studentID，second commit 我上傳 bf.m，final commit 我把剩下的部分都實作完成，跟改 bf.m 裡的 threshold 判斷。